实验 5 项目成本管理

1.实验目的

对项目进行成本估算，**功能点估算法是一个非常有用的，而且是国际通用的一种对软件规模进行估算的技术，**是项目管理人员必须掌握的工具。功能点估算法是在软件项目管理中项目计划制定的优劣直接关系到项目的成败，项目计划中对项目范围的估算又尤为重要，如果项目负责人对项目的规模没有一个比较客观的认识，没有对工作量、所需资源、完工时间等因素进行估算，那么项目计划也就没有存在的意义。

不要混淆报价和成本。

2.实验要求

提交实验为在线文档，文档地址为（实验老师规定），根据老师的要求提交文档，请按照目录分别存放。在线文档导出为本地文档，作为附件上传到智慧树，并同时贴出在线地址。

功能点估算：

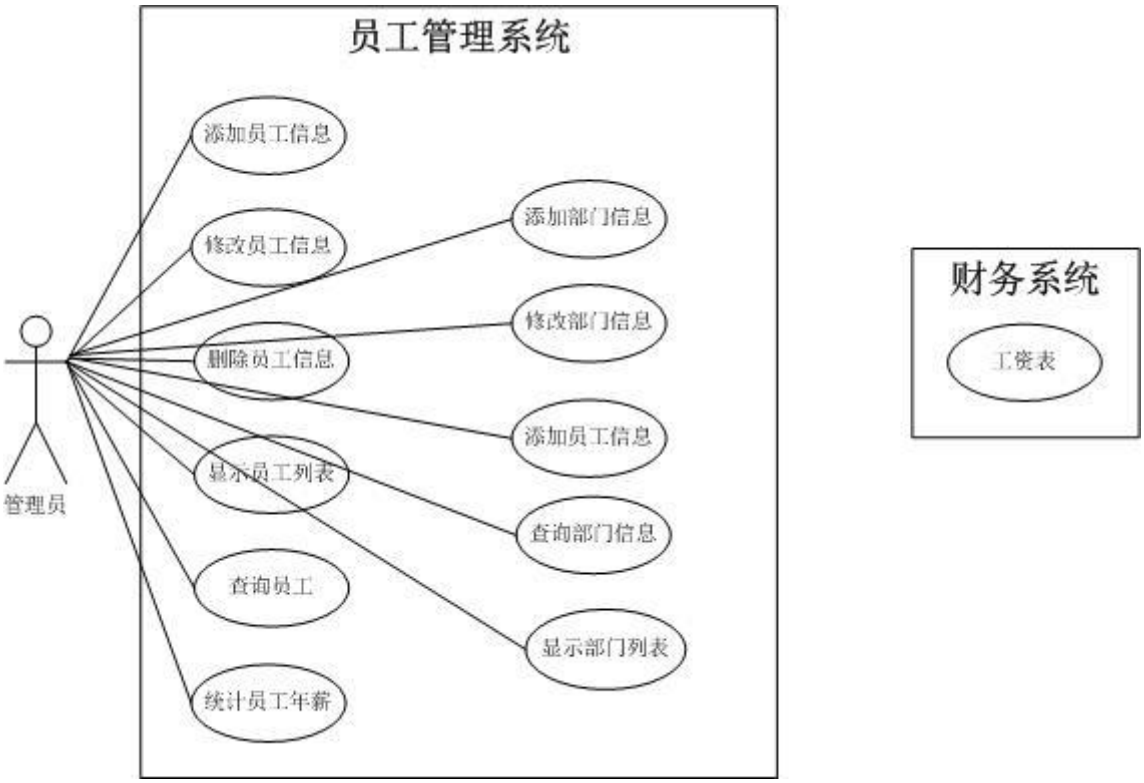
以《软件项目成本管理》为例子，进行项目成本估算。参考：《项目成本管理-界面设计》、《项目成本管理-用例设计》、《项目成本管理-数据库设计》。

1. 数据信息梳理。数据项目成本管理的所有数据字段信息。
2. 计算 ILF 和 EIF 功能点个数
3. 计算 EI EQ EO 功能点个数
4.

3. 实验内容-根据功能估算范例估算功能点

请同学们参考范例做出《项目成本管理》的功能点。

以员工管理系统为范例，在添加一个员工资料时会使用到员工的一般信息、员工的教育情况、工作经历和家属信息。员工又会隶属于某个部门，在本系统中会有一个对部门进行维护的功能。员工的工资是由另外一个财务系统提供的。因此其用例图如下所示：



1.1 数据信息梳理

1.1.1 假设员工基本信息如下所示：

员工 ID（标签控件）
员工名称
性别
生日
婚否
所属部门 ID（标签控件）
所属部门名称

——受教育的时间
——学校名称
——所学专业

——工作时间
——工作单位
——工作部门
——工作职务
——亲属的姓名
——之间关系
——亲属年龄
——工作单位

1.1.2 假设部门信息如下所示：

部门 ID（标签控件）
部门名称

1.1.3 假设工资表信息如下所示：

员工 ID（标签控件）
员工姓名
金额
单位

1.2 ILF 和 EIF 功能点个数

对 ILF 和 EIF 复杂度的计算可以简单理解为对数据库复杂度的计算。每个表单就是一个 ILF(内部逻辑文件)或 EIF（外部接口文件），表单上显示的字段都是 DET（数据单元类型 Data Element Type ），一个表单上的“核心”内容不管是由几个数据表来分别存放数据的，每个表都是一个 RET（记录单元类型（Record Element Type，RET ）。

简单来讲 ILF 和 EIF 可以被看作数据库中的数据表，但是主从表将被视为一个 ILF 或 EIF。那么 ILF 和 EIF 的复杂度就是由数据表中的字段 DET 和一个 ILF 或 EIF 自身所包含的主、从表个数 RET 来决定，但是在计算 DET 时主外键只能算作一个。

1. 数据单元类型（Data Element Type，DET ）：一个字段。用户可识别的无递归、

不重复的信息单元。

2. 记录单元类型（Record Element Type, RET ）：一条数据记录。在 ILF 或 EIF 中，用户可识别的数据域的子集，可以通过检查数据中的各种逻辑分组来识别它

参考资料 1：ILF EIF 定义（数据存储（数据库复杂度））

功能点	英文名字	描述
内部逻辑文件(ILF)	Internal Logical File	数据库的数据表数目
外部接口文件(EIF)	External Interface File	输入和输出之间的接口文件

参考资料 2：ILF/EIF 复杂度计算

记录单元	1~19 个 DET（数据单元）	20~50 个 DET	超过 51 个 DET
1 个 RET（	低	低	中等
2~5 个 RET	低	中等	高
6 个以上 RET	中等	高	高

参考资料 3：功能点计算：

功能点数量与复杂度对应矩阵，EI、EO、EQ、ILF 和 EIF 计算出来的技术复杂度对应的功能点如下表所示

	低	中等	高
EI	3 个功能点	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
EIF	5	7	10

计算结果：员工管理项目距 ILF 和 EIF 功能点数

ILF 内部逻辑文件	RET	DET 个数	复杂度	未调整的 FP 个数
员工信息	员工基本信息、员工受教育情况、工作经历、亲属信息，共 4 个。	18 个	低	7
部门信息	部门基本信息，共 1 个。	2 个	低	7
EIF 外部接口文件	**RET**	**DET 个数**	**复杂度**	**未调整的 FP 个数**
工资表	员工基本信息、工资信息，共 2 个。	4 个	低	5
合计：19 个				

1.3 EI、EQ 和 EO 功能点个数

EI 就是对应用户增加、修改、删除的操作，EO 和 EQ 都是用于用户查询的操作，但是 EO 和 EQ 的区别是 EO 查询时使用了数学公式或计算方法。EI、EQ 和 EO 的复杂度是由 FTR（引用文件类型（File Type Referenced,）和 DET 决定的。FTR 的个数是由 ILF 和 EIF 的个数决定的，可以由主表中主外键的个数来计算。

在计算 EI 的 DET 时，只有用户在界面上直接输入的信息才算作 DET，通过页面自动计算或转换的数据不能算作 EI 的 DET。在 EO 和 EQ 计算 DET 时，报表的标题、页码等信息不能被计算为一个 DET。

参考资料 4： EI EQ EO 定义

功能点	英文名字	描述

人机交互（程序复杂度）		
外部输入(EI)	External Input	数据输入界面
外部输出(EO)	External Output	主要指报表(Report)
外部查询(EQ)	External Inquiry	应用系统与外部之间的通信

参考资料 5：EI 复杂度计算矩阵

	1~4 个 DET	5~15 个 DET	多于 16 个 DET
0~1 个 FTR	低	低	中等
2 个 FTR	低	中等	高
大于 2 个 FRT	中等	高	高

参考资料 6：EO 和 EQ 复杂度计算矩阵

	1~5 个 DET	6~19 个 DET	多于 20 个 DET
0~1 个 FTR	低	低	中等
2~3 个 FTR	低	中等	高
多于 4 个 FTR	中等	高	高

计算结果:EI

EI	FTR	DET 个数	复杂度	未调整的 FP 个数
添加员工信息	员工、部门、工资表	员工信息的两个标签控件的内容不是用户输入的，因此不算。共 16 个。部门信息与员工信息中的部门字段重复，因此一个都不算。工资表中的员工 ID 和名称不能重复，因此只能算金额和单位，所以共 2 个 18 个	高	6
修改员工信息	员工、部	18 个同上	高	6

	门、工资表			
删除员工信息	员工、部门、工资表	1 个员工 ID	中等	4
添加部门信息	部门	1 个一个标签控件的内容不是用户输入的，因此不算	低	3
修改部门信息	部门	1 个一个标签控件的内容不是用户输入的，因此不算	低	3
删除部门息信	部门	1 个部门 ID	低	3
合计：25 个				

计算结果 EQ

EQ	FTR	DET 个数	复杂度	未调整的 FP 个数
查询员工信息	员工、部门、工资表	20 个	高	7
查询部门信息	部门	2 个	低	4
合计：11 个				

计算结果 EO

EO	FTR	DET 个数	复杂度	未调整的 FP 个数
统计员工年薪	员工、工资表	员工 ID、员工名称年份、年薪、单位共 5 个	低	4

1.4 调整因子计算

VAF	Value adjustment factor VAF=（sum of（DI）* 0.01）+ 0.65	功能点的调整因子的计算公式 VAF=（sum of（DI）* 0.01）+ 0.65

本系统的通用系统特性及其影响程度如下表所示：

VAF= 19 * 0.01 + 0.65 = 0.84

系统特性	分数
数据通讯	3
分布式数据处理	2
性能	0
高强度配置	0
交易速度	0
在线数据输入	5
最终用户效率	2
在线更新	3
复杂的处理	0
可复用性	3
易安装性	0
易操作性	0
多场地	0
支持变更	1
合计：19	
调整因子 = 19 * 0.01 + 0.65 = 0.84	

参考资料 7：VAF：计算调整因子的计算

功能点的调整因子系数是通过通用系统特性及其影响程度来评定的，对每个常规系统特性的评估由其影响程度（DI）而定，分为 0-5 级：

0 毫无影响

1 偶然影响

2　适度影响

3　一般影响

4　重要影响

5　强烈影响

然后依次对以下 14 个系统常规特性进行打分，并带入以下计算公式算出功能点的调整因子。Value Adjustment Factor=(sum of (DI) * 0.01) + 0.65

a. 数据通讯

数据通讯指的是应用程序直接与处理器通讯的程度。通常我们都是通过某种通讯手段来实现在一个应用中所使用的数据或者控制信息的。连接到本地控制器上的终端被认为是使用通讯设施，而协议指的是两个系统或者两个设备之间进行通讯时所使用的一种约定。所有的数据通讯链接都需要某种协议。

0	应用程序是单纯的批处理或者 PC stand-alone
1	应用程序是一种批处理过程，但是包含远程数据的录入或远程打印
2	应用程序是一种批处理过程，但是包含远程数据的录入和远程打印
3	应用程序包括在线数据收集或者包括批处理或查询系统的远程处理的前端应用
4	应用程序不单只是前端应用，但是仅支持一种远程处理通讯协议
5	应用程序不单只是前端应用，还支持多于一种的远程处理通讯协议

b. 2. 分布式数据处理

分布式数据处理是应用在内部组件之间传递信息的程度。这个特性是在应用边界内体现的。

0	应用程序不支持组件之间的数据传输和处理功能
1	应用程序为用户可能进行的处理准备数据（例如使用电子表格或者数据库等）
2	应用程序所准备的数据是为了在系统另外一个组件上传输和处理。并非为终端用户所处理。
3	分布式处理和数据传输是在线的，并且是单向的
4	分布式处理和数据传输是在线的，并且是双向的
5	由系统中最恰当的组件动态地执行处理功能

c. 3. 性能

性能是吞吐量、处理时间等指标对开发的影响。用户所提出的性能要求将直接影响到系统的设计，实施，安装和支持。

0	用户没有提出性能方面的要求
1	用户提出了性能和设计方面的要求，但不需要采取特定措施
2	响应时间和吞吐量在系统峰值时是关键的，但是不需要采取相应的 CPU 使用方面的特殊设计。处理的最后期限是在下一个工作日。
3	在任何时候响应时间和吞吐量都是关键的，但是不需要采取相应的 CPU 使用方面的特殊设计。处理的完成期限比较严格
4	除了上面一项的要求外，由于对需求的要求比较严格，在设计阶段就要进行性能分析
5	除了上面一项的要求之外，在设计和实施阶段需要使用性能分析工具来判断性能要求的完成情况

d. 4.大业务量配置

大业务量配置指的是计算机的资源对应用开发的影响程度。大业务量的运行配置对设计有特殊要求，是必须考虑的一个系统特性。

0	没有提出明确的运行方面的限制
1	有运行方面的限制，但是不需要采取特别的措施以满足运行限制
2	提出了一些安全和时间方面的限制
3	应用程序的某些部分对处理器有特定的要求。
4	提出的运行限制对应用的中央处理器或者专用处理器有特殊的要求
5	除上面一项之外，还对应用的分布式组件提出了限制

e. 5.事务处理率

事务处理率是业务交易处理速度的要求对系统的设计，实施，安装和支持等的影响。

0	预计不会出现周期性的高峰事务处理期
1	预计会有周期性的高峰事务处理期（例如：每月、每季、每年）
2	预计每周都会出现高峰事务处理期

3	预计每天都会出现高峰事务处理期
4	用户在应用程序需求或者服务级别协议中对事务率要求很高，因此必须在设计阶段进行性能分析。
5	用户在应用程序需求或者服务级别协议中对事务率要求很高，因此必须进行性能分析并在设计、开发和安装阶段中使用到性能分析工具。

f. 6.在线数据输入

在线数据输入是指数据通过交互的方式输入系统程度。系统中包括在线数据输入和控制信息功能。

0	所有事务都是批处理的。
1	1%~7%的事务是以交互式的方式进行数据录入
2	8%~15%的事务是以交互式的方式进行数据录入
3	16%~23%的事务是以交互式的方式进行数据录入
4	24%~30%的事务是以交互式的方式进行数据录入
5	30%以上的事务是以交互式的方式进行数据录入

g. 7.最终用户效率

最终用户效率是指对应用的人文因素以及使用的便捷方面的考虑程度。如下功能设计是针对最终用户效率的：

页面导航

菜单

在线帮助或文档

光标自动跳转

可以滚动

在线远程打印

预定义的功能键

在线做批量提交任务

光标可以选取界面上的数据

用户使用大量反白显示、重点显示、下划线或其他的标识

在线 copy 用户文档

鼠标拖动功能

弹出窗体

使用最少的界面完成某种商业功能

双语言支持（如果选择了这个就算 4 项）

多语言支持（如果选择了这个就算 6 项）

0	以上的一个都不包括
1	包括以上的 1~3 个
2	包括以上的 4~5 个
3	包括以上的 6 个或以上，但是没有用户对于效率的要求
4	包括以上的 6 个或以上，对用户使用效率有较高要求，因而必须考虑用户方面的设计（例如，最少击键次数、尽可能提供默认值、模版的使用）
5	包括以上的 6 个或以上，用户对效率的要求使得开发人员必须使用特定的工具和流程以判定用户对效率的要求已经被达成

h. 8. 在线更新

在线更新是指内部逻辑文件 ILF 被在线更新的程度。应用系统提供在线更新内部逻辑文件的功能。

0	没有在线更新
1	包含 1~3　个控制文件的在线更新。更新的流量低，恢复容易
2	包含对 4　个以上控制文件的在线更新。更新的流量低，恢复容易
3	包含对主要 ILF　的更新
4	除了 3　之外，在设计和实施中要考虑对数据丢失的防范。
5	除了 4　之外，大量的数据恢复工作要考虑成本因素，同时包含了高度自动化的恢复流程。

i. 9. 复杂处理

复杂处理描述了逻辑处理对应用开发的影响程度。　它包含以下要素：

敏感控制（例如特殊的审核过程）和/或程序特定的安全处理

大量的逻辑处理

大量的数学处理

因为例外处理造成的需要重新处理的情况（例如，由 TP 中断、数据值缺少和验证失败导致的 ATM 事务）

多种可能的输入/输出造成的复杂处理

0	上面一个都不满足
1	只满足一个
2	只满足两个
3	满足三个
4	满足四个
5	都满足

j. 10. 可复用性

应用系统中的应用和代码经过特殊设计、开发和支持，可以在其他应用系统中复用。

0	没有可复用的代码
1	代码在应用之内复用
2	应用中被其他用户复用的部分不足 10%
3	应用中被不止一个用户使用的部分超过 10%
4	应用遵从一种易于复用的方式被打包和文档化。用户在源代码级客户化该应用
5	应用按照一种易于复用的方式被打包和文档化。用户使用用户参数来对该应用进行客户化

k. 12.易安装性

易安装性指应用系统的转换和安装容易度对开发的影响程度。系统测试阶段提供了转换和安装计划和/或转换工具。

0	用户对安装没有特定的要求
1	用户对安装没有特定的要求，但有特定的安装环境要求
2	用户提出了安装和转化的要求，转化/安装指南被经过测试提供给用户。但是转化的影响对该应用不重要
3	用户提出了安装和转化的要求，转化/安装指南被经过测试提供给用户。转化的影响对该应用来说是重要的

4	除了 2　的要求之外，需要提供经过测试的自动化的安装和转化工具。
5	除了 3　的要求之外，需要提供经过测试的自动化的安装和转化工具。

l.　12. 易操作性

易操作性指的是应用对运行的影响程度，如有效启动、备份和恢复规程的影响。易操作性是应用提供的一种特性，它最小化了手工操作的要求。

0	用户没有指定除正常备份程序外的其它特定操作
1	提供高效的启动、备份和恢复进程，但需要人手操作
2	提供高效的启动、备份和恢复进程，不需要人手操作（当作两项计算）
3	应用程序对磁带的需求最小化
4	应用程序对硬拷贝处理的需求最小化
5	程序设计成无人操作模式。无人操作模式的意思是除了启动和关闭之外，不需要对系统进行操作。程序的其中一个功能就是错误自动恢复。

m. 13. 多场地

多场地指应用系统经特殊设计、开发可以在多个组织、多个地点应用的程度。

0	用户需求不含多场地和组织的要求
1	考虑了多场地的要求，但是设计要求应用在不同的场地使用相同的软硬件环境
2	考虑了多场地的要求，但是设计要求应用在不同的场地使用类似的软硬件环境
3	考虑了多场地的要求，同时设计支持应用在不同的场地使用不同的软硬件环境
4	在 1　或者 2　的要求之上，提供了经过测试的多场地的文档和支持计划
5	在 3　的要求之上，提供了经过测试的多场地的文档和支持计划

2. 计算结果：　调整后的功能点数

功能点的原始计算公式：FP Count =UFP * VAF　=

（19 + 25 + 11 + 5）* 0.84 =50.4 个

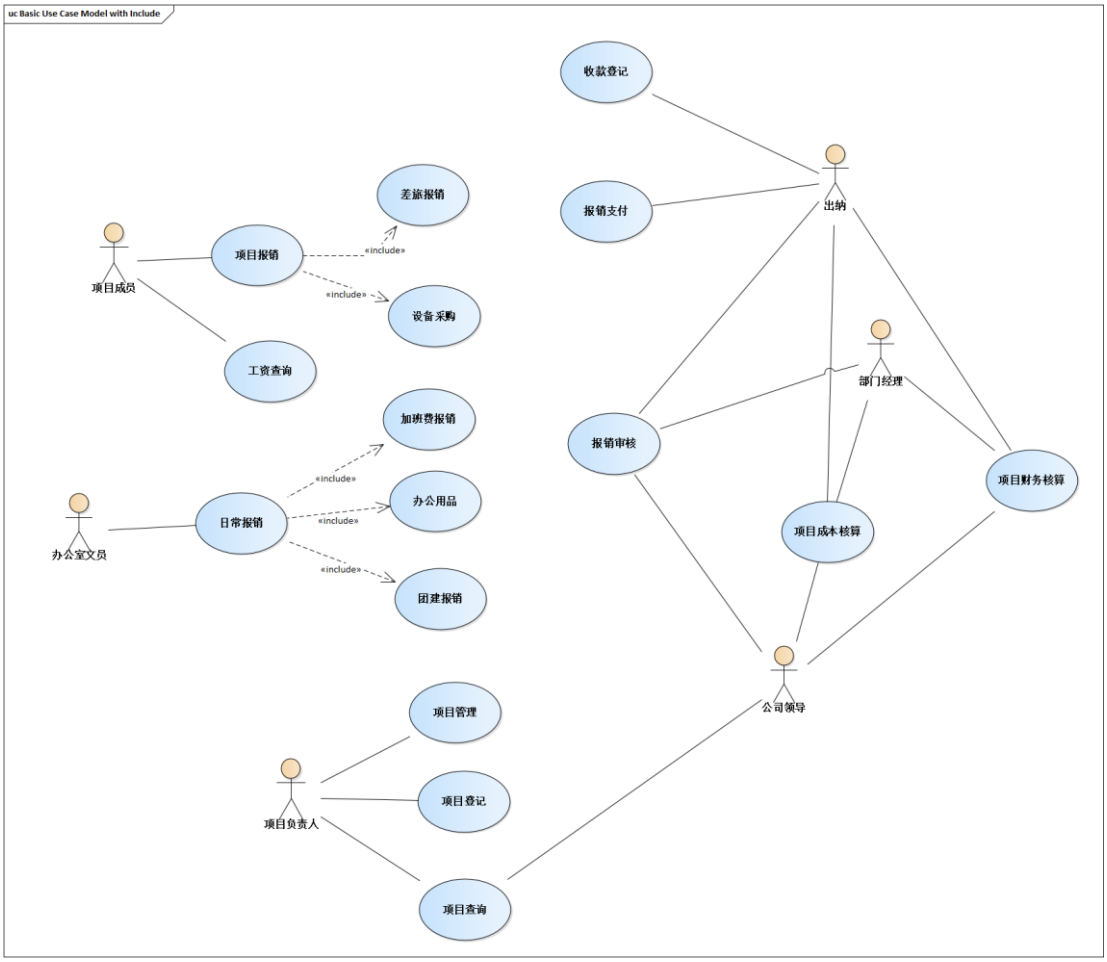
UFP	Unadjusted functional point count	未调整的功能点个数
VAF	Value adjustment factor VAF=（sum of（DI）* 0.01）+ 0.65	功能点的调整因子的计算公式 VAF=（sum of（DI）* 0.01）+ 0.65

4. 实验内容-挣值分析-杨磊

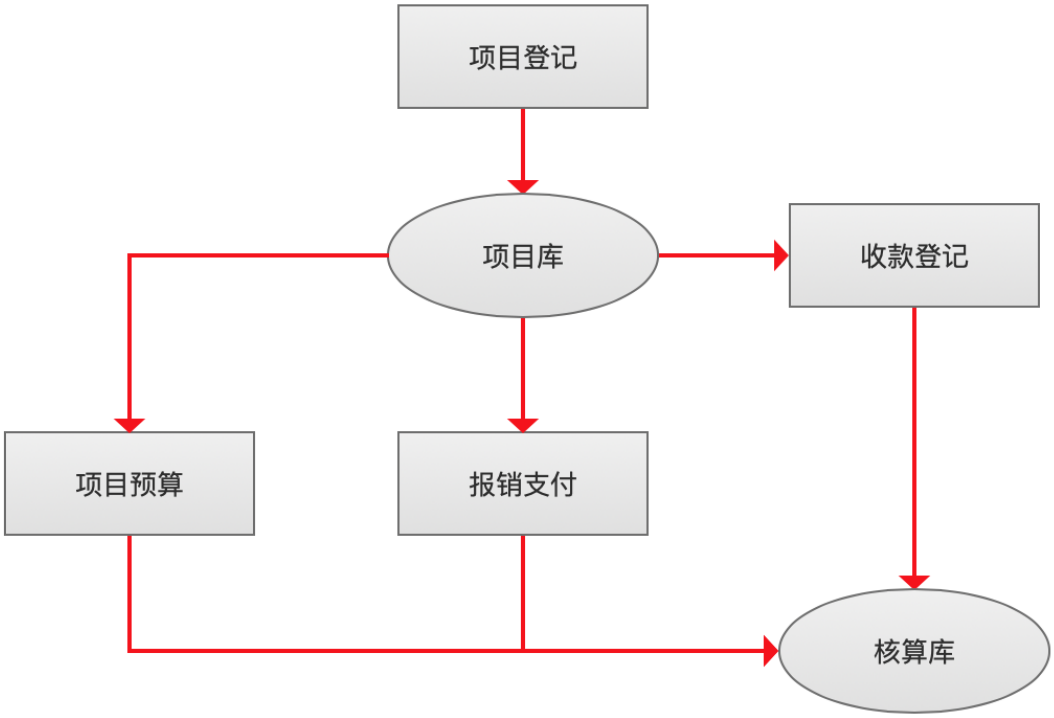
关键技术-挣值分析-杨磊.pptx

附录 1：项目成本管理案例

1. 项目成本管理-用例设计



2. 项目成本管理-界面设计



项目登记：

项目名称　　负责人

添加　删除　编辑

编号	项目名称	合同对方	签订时间	合同金额（元）	计划完成时间	已收款	已收款比例	项目负责人
1	山大焊接项目	山东大学（陈迹）	2019年4月1日	158000.00	2019年4月30日	40000.00	50%	宋冰飞
2								

收款登记：

项目名称 山大焊接项目　　负责人 张朝阳

添加　删除　编辑

	收款日期	收款方式	收款摘要	收入金额	是否开票	备注
1	2019年7月20日	电汇	第一次付款	40000.00	已开专票后作废	委托第三方支付
2	2019年9月2日	电汇	第二次付款	20000.00	未开发票	第三方支付

报销支付：

费　用　报　销　单

单据及附件共　　项

年　月　日

报销类别 公共费用报销

序号	报销科目	摘要	金额	备注
1	快递费	快递发票	18.00	
2	电话费	7-8月份固话话费	295.00	
3	办公费	购买鼠标	129.00	
4	福利费	误餐补助	1520.00	
5	财务费	2季度会计代理费	1800.00	
	合计			

全额大写：　拾　万　仟　佰　拾　元　角　分　原借款：　元　应退（补）款：　元

公司领导　　项目负责人　　出纳　　报销人

项目财务核算：

项目名称 山大焊接项目　　负责人 张朝阳

查询

	日期	科目	摘要	收入金额	支出金额	余额	备注
1	20180801	项目款	项目一期款	40000.00	0.00	40000.00	
2	20190805	差旅费	出差北京	0.00	1000.00	39000.00	
3	20190810	打印复印费	打印说明书	0.00	100.00	38900.00	

项目全成本核算：

项目名称 山大焊接项目　　负责人 张朝阳

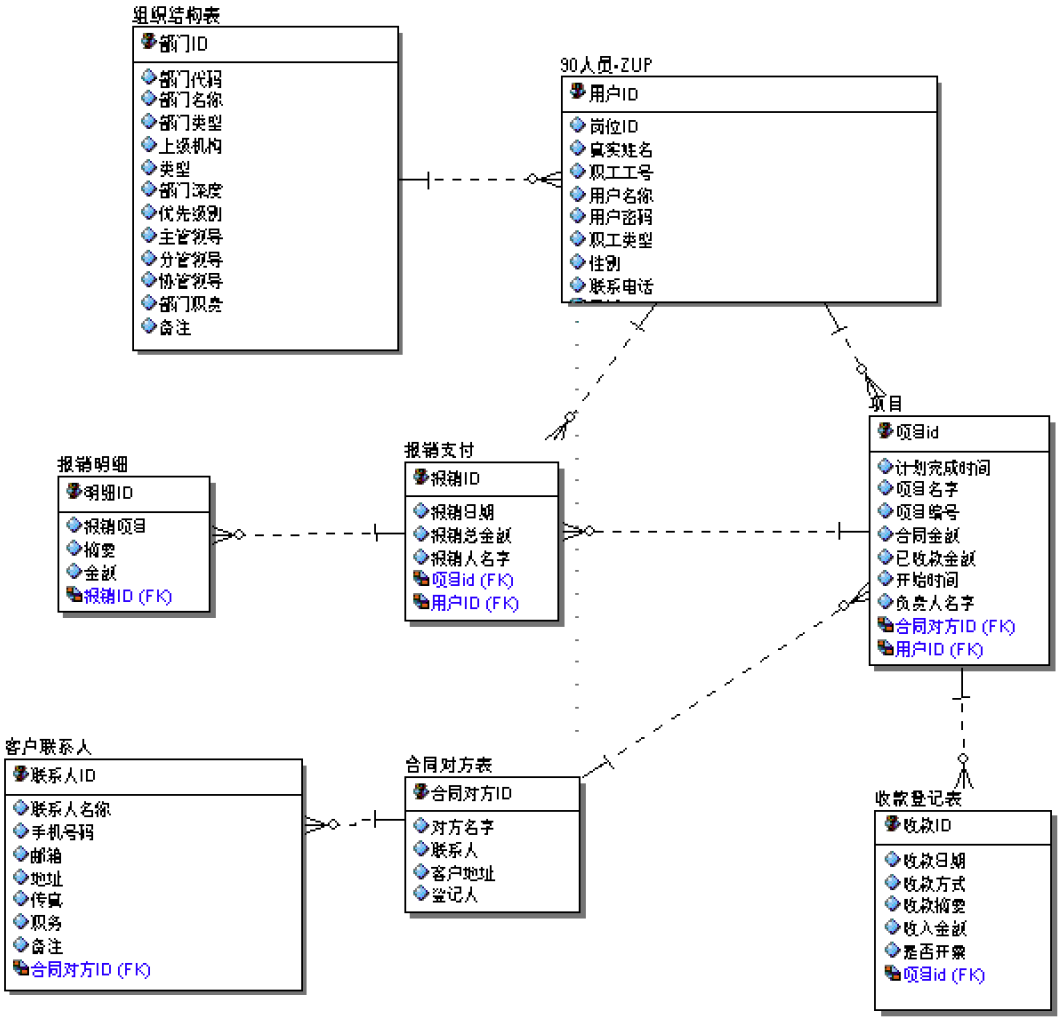
编号	科目	子科目	单价	数量	金额	工作事项	备注
1	收入	项目收入	158000.00	1	158000.00		全款
2	成本	税费	158000.00	10%	=单价*数量	税金和管理费	
		甲外委费	1000.00	20	=单价*数量		
		乙外协费	700.00	20	=单价*数量		
		丙外协费	500.00	10	=单价*数量		
		内部甲人工费					
		内部乙人工费					
		内部丙人工费					
		差旅费					
		营销费					
		其他					
3	利润	=收入-成本					

现金流水：

年份 月份 核算表

	日期	摘要	收入金额	还借款	支出金额	借款	余额	备注
1	20190801	取备用金	50000.00	0.00	0.00	0.00	50000.00	
2	20190810	工资	0.00	0.00	10000.00	0.00	40000.00	
3	20190812	徐惠三借款	0.00	0.00	0.00	3000.00	37000.00	
4	20190815	徐惠三还借款	0.00	3000.00	0.00	0.00	40000.00	
5	20190815	徐惠三报销差旅费	0.00	0.00	2500.00	0.00	37500.00	
	...							
合计								

3. 数据库设计



附录 2：功能点估算参考资料

功能点估算法是一个非常有用的，而且是国际通用的一种对软件规模进行估算的技术，软件项目管理众多知识中比较有技术含量的一个，是项目管理人员必须掌握的工具。

功能点估算法是在软件项目管理中项目计划制定的优劣直接关系到项目的成败，项目计划中对项目范围的估算又尤为重要，如果项目负责人对项目的规模没有一个比较客观的认识，没有对工作量、所需资源、完工时间等因素进行估算，那么项目计划也就没有存在的意义。

项目范围的估算在 CMMI 的“MA”度量分析管理和“PP”项目计划中均有涉及，对软件项目成本的估算有很多种方法，常见的就是 LOC 代码行和 FP 功能点法，它们之间的区别和关系如下：

1、FP 功能点估算法常用在项目开始或项目需求基本明确时使用，这时进行估算其结果的准确性比较高，假如这个时候使用 LOC 代码行估算法，则误差会比较大。

2、FP 功能点法是以用户为角度进行估算，LOC 代码行估算法则是以技术为角度进行估算的。使用 FP 功能点估算法无需懂得软件使用何种开发技术。LOC 代码行估算法与软件开发技术密切相关。

3、通过一些行业标准或企业自身度量的分析，FP 功能点估算法是可以转换为 LOC 代码行的。在项目刚开始的时候进行功能点估算可以对项目的范围进行预测，在项目开发的过程中由于**需求的变更和细化可能会导致项目范围的蔓延**，计算出来的结果会与当初估计的不同，因此在项目结束时还需要对项目的范围情况进行估算，这个时候估算的结果才能最准确反映项目的规模。

2.1 功能点估算的步骤

1、 识别功能点的类型。

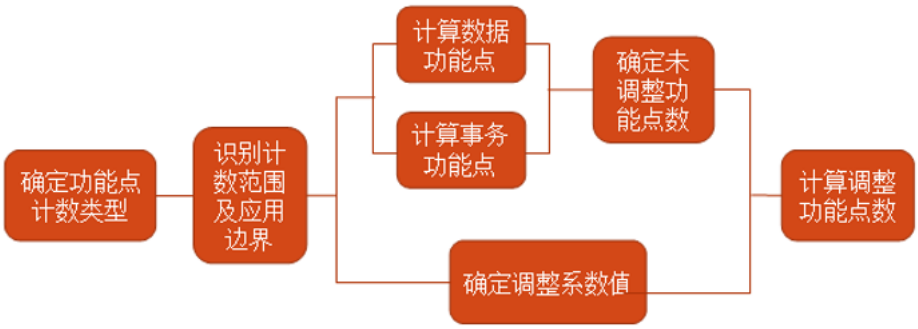
2、 识别待估算应用程序的边界和范围。

3、 计算**数据类型功能点**所提供的未调整的功能点数量。

4、 计算**人机交互功能**所提供的未调整的功能点数量。

5、 确定调整因子。

6、 计算调整后的功能点数量。



2.2.1 功能点的原始计算公式

功能点的原始计算公式：FP Count =UFP * VAF

UFP	Unadjusted functional point count	未调整的功能点个数
VAF	Value adjustment factor VAF=（sum of（DI）* 0.01）+ 0.65	功能点的调整因子的计算公式 VAF=（sum of（DI）* 0.01）+ 0.65

DI： 影响程度

2.2.2 计算 EI/EO/EQ/EIF/LIF

由于计算机软件就是为了实现无纸办公，那么在估算功能点时应该多以用户的纸质表单为依据。

每个表单就是一个 ILF(内部逻辑文件)或 EIF（外部接口文件），表单上显示的字段都是 DET（数据单元类型 Data Element Type ），一个表单上的“核心”内容不管是由几个数据表来分别存放数据的，每个表都是一个 RET（记录单元类型

（Record Element Type, RET ）。简单来讲 ILF 和 EIF 可以被看作数据库中的数据表，但是主从表将被视为一个 ILF 或 EIF。那么 ILF 和 EIF 的复杂度就是由数据表中的字段 DET 和一个 ILF 或 EIF 自身所包含的主、从表个数 RET 来决定，但是在计算 DET 时主外键只能算作一个。

EI 就是对应用户增加、修改、删除的操作，EO 和 EQ 都是用于用户查询的操作，但是 EO 和 EQ 的区别是 EO 查询时使用了数学公式或计算方法。EI、EQ 和 EO 的复杂度是由 FTR（引用文件类型（File Type Referenced,）和 DET 决定的。FTR 的个数是由 ILF 和 EIF 的个数决定的，可以由主表中主外键的个数来计算。

在计算 EI 的 DET 时，只有用户在界面上直接输入的信息才算作 DET，通过页面自动计算或转换的数据不能算作 EI 的 DET。在 EO 和 EQ 计算 DET 时，报表的标题、页码等信息不能被计算为一个 DET。

ILF、EIF 要与 EI、EO、EQ 分开计算。对 ILF 和 EIF 复杂度的计算可以简单理解为对数据库复杂度的计算。对 EI、EO、EQ 复杂度的计算可以理解为对程序开发复杂度的计算。一般软件项目都是由数据和程序构成的，因此计算 ILF、EIF 和计算 EI、EO、EQ 之间没有任何关系。

功能点	英文名字	描述
人机交互（程序复杂度）		
外部输入(EI)	External Input	数据输入界面
外部输出(EO)	External Output	主要指报表(Report)
外部查询(EQ)	External Inquiry	应用系统与外部之间的通信
数据存储（数据库复杂度）		
内部逻辑文件(ILF)	Internal Logical File	数据库的数据表数目
外部接口文件(EIF)	External Interface File	输入和输出之间的接口文件

参数介绍：

1. 数据单元类型（Data Element Type, DET ）：一个字段

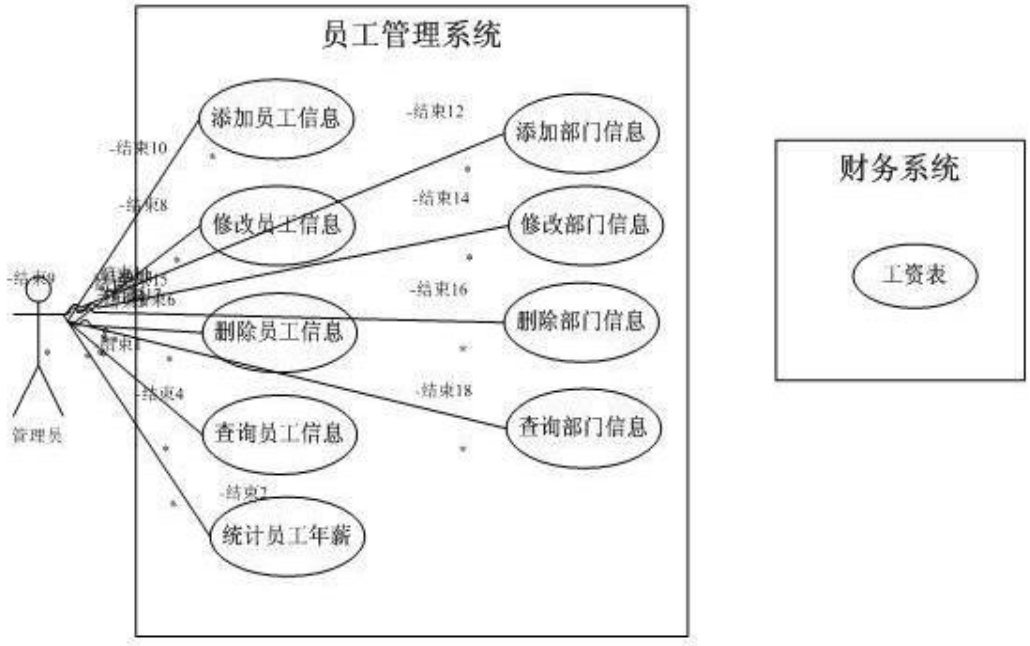
用户可识别的无递归、不重复的信息单元。

2. 记录单元类型（Record Element Type, RET ）：一条数据记录

在 ILF 或 EIF 中，用户可识别的数据域的子集，可以通过检查数据中的各种逻辑分组来识别它

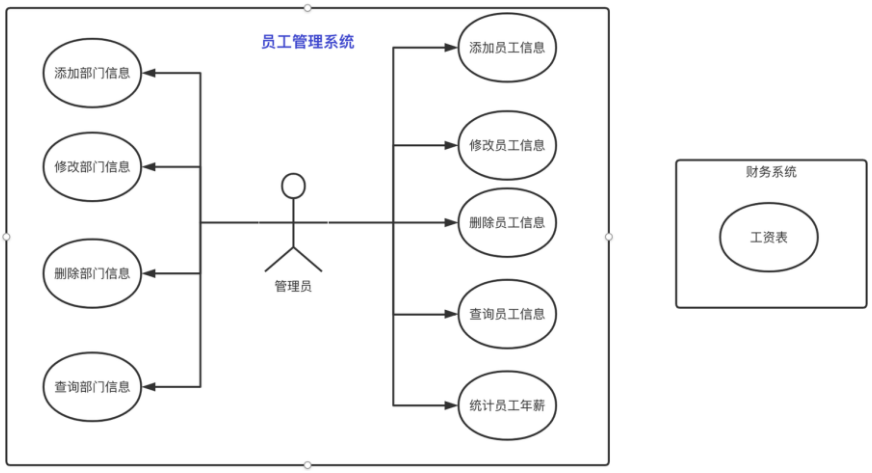
3. 引用文件类型（ File Type Referenced, FTR ）

被事务功能读取或维护的内部逻辑文件(ILF)，或者是被事务功能读取的外部接口文件(EIF)。



【腾讯文档】员工管理系统例子

[员工管理系统例子]



计算 EIF 和 LIF:

ILF 内部逻辑文件	**RET** **（数据表）**	**DET 个数** **（字段）**	**复杂度**	**未调整的 FP 个数**
员工信息	员工基本信息、员工受教育情况、工作经历、亲属信息，共 4 个。	18 个	低	7
部门信息	部门基本信息，共 1 个。	2 个	低	7

EIF 外部接口文件	**RET**	**DET 个数**	**复杂度**	**未调整的 FP 个数**
工资表	员工基本信息、工资信息，共 2 个。	4 个	低	5
合计：19 个				

计算 EI/EO/EQ

EI	**FTR**	**DET 个数**	**复杂度**	**未调整的 FP 个数**
添加员工信息	员工、部门、工资表	员工信息的两个标签控件的内容不是用户输入的，因此不算。共 16 个。部门信息与员工信息中的部门字段重复，因此一个都不算。工资表中的员工 ID 和名称	高	6

		不能重复，因此只能算金额和单位，所以共 2 个 18 个		
修改员工信息	员工、部门、工资表	18 个同上	高	6
删除员工信息	员工、部门、工资表	1 个员工 ID	中等	4
添加部门信息	部门	1 个一个标签控件的内容不是用户输入的，因此不算	低	3
修改部门信息	部门	1 个一个标签控件的内容不是用户输入的，因此不算	低	3
删除部门息信	部门	1 个部门 ID	低	3
合计：25 个				
EQ	**FTR**	**DET 个数**	**复杂度**	**未调整的 FP 个数**
查询员工信息	员工、部门、工资表	20 个	高	7
查询部门信息	部门	2 个	低	4
合计：11 个				
EO	**FTR**	**DET 个数**	**复杂度**	**未调整的 FP 个数**
统计员工年薪	员工、工资表	员工 ID、员工名称年份、年薪、单位共 5 个	低	4

2.2.3 基于复杂度估算功能点数量

EI 复杂度计算矩阵

	1~4 个 DET	5~15 个 DET	多于 16 个 DET
0~1 个 FTR	低	低	中等

2 个 FTR	低	中等	高
大于 2 个 FRT	中等	高	高

EO 和 EQ 复杂度计算矩阵

	1~5 个 DET	6~19 个 DET	多于 20 个 DET
0~1 个 FTR	低	低	中等
2~3 个 FTR	低	中等	高
多于 4 个 FTR	中等	高	高

ILF/EIF 复杂度的矩阵

记录单元	1~19 个 DET（数据单元）	20~50 个 DET	超过 51 个 DET
1 个 RET（	低	低	中等
2~5 个 RET	低	中等	高
6 个以上 RET	中等	高	高

功能点计算：

功能点数量与复杂度对应矩阵，EI、EO、EQ、ILF 和 EIF 计算出来的技术复杂度对应的功能点如下表所示

	低	中等	高
EI	3 个功能点	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
EIF	5	7	10

复杂性取决于 FTRs 和 DETs 的数量。FTR 是被一个事物操作读取或维护的一个 ILF，或者是被一个事物操作读取的一个 EIF。

根据复杂度判断功能点数，下边为检索表。

<table>
<tr><td rowspan="2">类别</td><td rowspan="2">功能点</td><td rowspan="2">涉及的文件类型
FTR(files type Referrenced)</td><td colspan="3">数据元(Data Elements)</td></tr>
<tr><td>1～4</td><td>5～15</td><td>大于 15</td></tr>
<tr><td rowspan="12">人机交互（程序复杂度）</td><td rowspan="3">EI</td><td>少于 2 种</td><td>低（3）</td><td>低（3）</td><td>普通（4）</td></tr>
<tr><td>2</td><td>低（3）</td><td>普通（4）</td><td>高（6）</td></tr>
<tr><td>多于 2 种</td><td>普通（4）</td><td>高（6）</td><td>高（6）</td></tr>
<tr><td rowspan="5">EO</td><td rowspan="2">涉及的文件类型
FTR(files type Referrenced)</td><td colspan="3">数据元(Data Elements)</td></tr>
<tr><td>1～5</td><td>6～19</td><td>大于 19</td></tr>
<tr><td>少于 2 种</td><td>低（4）</td><td>低（4）</td><td>普通（5）</td></tr>
<tr><td>2</td><td>低（4）</td><td>普通（5）</td><td>高（7）</td></tr>
<tr><td>多于 2 种</td><td>普通（5）</td><td>高（7）</td><td>高（7）</td></tr>
<tr><td rowspan="5">EQ</td><td rowspan="2">涉及的文件类型
FTR(files type Referrenced)</td><td colspan="3">数据元(Data Elements)</td></tr>
<tr><td>1～5</td><td>6～19</td><td>大于 19</td></tr>
<tr><td>少于 2 种</td><td>低（4）</td><td>低（4）</td><td>普通（5）</td></tr>
<tr><td>2</td><td>低（4）</td><td>普通（5）</td><td>高（7）</td></tr>
<tr><td>多于 2 种</td><td>普通（5）</td><td>高（7）</td><td>高（7）</td></tr>
<tr><td rowspan="10">数据存储（数据库复杂度）</td><td rowspan="5">ILF</td><td rowspan="2">记录元素类型
RET(record element types)</td><td colspan="3">数据元(Data Elements)</td></tr>
<tr><td>1～19</td><td>20～50</td><td>大于 51</td></tr>
<tr><td>1</td><td>低（7）</td><td>低（7）</td><td>普通（10）</td></tr>
<tr><td>2~5</td><td>低（7）</td><td>普通（10）</td><td>高（15）</td></tr>
<tr><td>6 种以上</td><td>普通（10）</td><td>高（15）</td><td>高（15）</td></tr>
<tr><td rowspan="5">ELF</td><td rowspan="2">记录元素类型
RET(record element types)</td><td colspan="3">数据元(Data Elements)</td></tr>
<tr><td>1～19</td><td>20～50</td><td>大于 51</td></tr>
<tr><td>1</td><td>低（7）</td><td>低（7）</td><td>普通（10）</td></tr>
<tr><td>2~5</td><td>低（7）</td><td>普通（10）</td><td>高（15）</td></tr>
<tr><td>6 种以上</td><td>普通（10）</td><td>高（15）</td><td>高（15）</td></tr>
</table>

2.2.4 VAF：计算调整因子的计算

功能点的调整因子系数是通过通用系统特性及其影响程度来评定的，对每个常规系统特性的评估由其影响程度（DI）而定，分为 0-5 级：

0　毫无影响

1　偶然影响

2　适度影响

3　一般影响

4　重要影响

5　强烈影响

然后依次对以下 14 个系统常规特性进行打分，并带入以下计算公式算出功能点的调整因子。Value Adjustment Factor=(sum of (DI) * 0.01) + 0.65

1. 数据通讯

数据通讯指的是应用程序直接与处理器通讯的程度。通常我们都是通过某种通讯手段来实现在一个应用中所使用的数据或者控制信息的。连接到本地控制器上的终端被认为是使用通讯设施，而协议指的是两个系统或者两个设备之间进行通讯时所使用的一种约定。所有的数据通讯链接都需要某种协议。

0	应用程序是单纯的批处理或者 PC stand-alone
1	应用程序是一种批处理过程，但是包含远程数据的录入或远程打印
2	应用程序是一种批处理过程，但是包含远程数据的录入和远程打印
3	应用程序包括在线数据收集或者包括批处理或查询系统的远程处理的前端应用
4	应用程序不单只是前端应用，但是仅支持一种远程处理通讯协议
5	应用程序不单只是前端应用，还支持多于一种的远程处理通讯协议

2. 分布式数据处理

分布式数据处理是应用在内部组件之间传递信息的程度。这个特性是在应用

边界内体现的。

0	应用程序不支持组件之间的数据传输和处理功能
1	应用程序为用户可能进行的处理准备数据（例如使用电子表格或者数据库等）
2	应用程序所准备的数据是为了在系统另外一个组件上传输和处理。并非为终端用户所处理。
3	分布式处理和数据传输是在线的，并且是单向的
4	分布式处理和数据传输是在线的，并且是双向的
5	由系统中最恰当的组件动态地执行处理功能

3. 性能

性能是吞吐量、处理时间等指标对开发的影响。用户所提出的性能要求将直接影响到系统的设计，实施，安装和支持。

0	用户没有提出性能方面的要求
1	用户提出了性能和设计方面的要求，但不需要采取特定措施
2	响应时间和吞吐量在系统峰值时是关键的，但是不需要采取相应的 CPU 使用方面的特殊设计。处理的最后期限是在下一个工作日。
3	在任何时候响应时间和吞吐量都是关键的，但是不需要采取相应的 CPU 使用方面的特殊设计。处理的完成期限比较严格
4	除了上面一项的要求外，由于对需求的要求比较严格，在设计阶段就要进行性能分析
5	除了上面一项的要求之外，在设计和实施阶段需要使用性能分析工具来判断性能要求的完成情况

4.大业务量配置

大业务量配置指的是计算机的资源对应用开发的影响程度。大业务量的运行配置对设计有特殊要求，是必须考虑的一个系统特性。

0	没有提出明确的运行方面的限制
1	有运行方面的限制，但是不需要采取特别的措施以满足运行限制
2	提出了一些安全和时间方面的限制

3	应用程序的某些部分对处理器有特定的要求。
4	提出的运行限制对应用的中央处理器或者专用处理器有特殊的要求
5	除上面一项之外，还对应用的分布式组件提出了限制

5.事务处理率

事务处理率是业务交易处理速度的要求对系统的设计，实施，安装和支持等的影响。

0	预计不会出现周期性的高峰事务处理期
1	预计会有周期性的高峰事务处理期（例如：每月、每季、每年）
2	预计每周都会出现高峰事务处理期
3	预计每天都会出现高峰事务处理期
4	用户在应用程序需求或者服务级别协议中对事务率要求很高，因此必须在设计阶段进行性能分析。
5	用户在应用程序需求或者服务级别协议中对事务率要求很高，因此必须进行性能分析并在设计、开发和安装阶段中使用到性能分析工具。

6.在线数据输入

在线数据输入是指数据通过交互的方式输入系统程度。系统中包括在线数据输入和控制信息功能。

0	所有事务都是批处理的。
1	1%~7%的事务是以交互式的方式进行数据录入
2	8%~15%的事务是以交互式的方式进行数据录入
3	16%~23%的事务是以交互式的方式进行数据录入
4	24%~30%的事务是以交互式的方式进行数据录入
5	30%以上的事务是以交互式的方式进行数据录入

7.最终用户效率

最终用户效率是指对应用的人文因素以及使用的便捷方面的考虑程度。如下功能设计是针对最终用户效率的：

页面导航

菜单

在线帮助或文档

光标自动跳转

可以滚动

在线远程打印

预定义的功能键

在线做批量提交任务

光标可以选取界面上的数据

用户使用大量反白显示、重点显示、下划线或其他的标识

在线 copy 用户文档

鼠标拖动功能

弹出窗体

使用最少的界面完成某种商业功能

双语言支持（如果选择了这个就算 4 项）

多语言支持（如果选择了这个就算 6 项）

0	以上的一个都不包括
1	包括以上的 1~3 个
2	包括以上的 4~5 个
3	包括以上的 6 个或以上，但是没有用户对于效率的要求
4	包括以上的 6 个或以上，对用户使用效率有较高要求，因而必须考虑用户方面的设计（例如，最少击键次数、尽可能提供默认值、模版的使用）
5	包括以上的 6 个或以上，用户对效率的要求使得开发人员必须使用特定的工具和流程以判定用户对效率的要求已经被达成

8. 在线更新

在线更新是指内部逻辑文件 ILF 被在线更新的程度。应用系统提供在线更新内部逻辑文件的功能。

0	没有在线更新
1	包含 1~3　个控制文件的在线更新。更新的流量低，恢复容易
2	包含对 4　个以上控制文件的在线更新。更新的流量低，恢复容易
3	包含对主要 ILF　的更新

4	除了 3　之外，在设计和实施中要考虑对数据丢失的防范。
5	除了 4　之外，大量的数据恢复工作要考虑成本因素，同时包含了高度自动化的恢复流程。

9. 复杂处理

复杂处理描述了逻辑处理对应用开发的影响程度。　它包含以下要素：

敏感控制（例如特殊的审核过程）和/或程序特定的安全处理

大量的逻辑处理

大量的数学处理

因为例外处理造成的需要重新处理的情况（例如，由 TP 中断、数据值缺少和验证失败导致的 ATM 事务）

多种可能的输入/输出造成的复杂处理

0	上面一个都不满足
1	只满足一个
2	只满足两个
3	满足三个
4	满足四个
5	都满足

10. 可复用性

应用系统中的应用和代码经过特殊设计、开发和支持，可以在其他应用系统中复用。

0	没有可复用的代码
1	代码在应用之内复用
2	应用中被其他用户复用的部分不足 10%
3	应用中被不止一个用户使用的部分超过 10%
4	应用遵从一种易于复用的方式被打包和文档化。用户在源代码级客户化该应用
5	应用按照一种易于复用的方式被打包和文档化。用户使用用户参数来对该应用进行客户化

12.易安装性

易安装性指应用系统的转换和安装容易度对开发的影响程度。系统测试阶段提供了转换和安装计划和/或转换工具。

0	用户对安装没有特定的要求
1	用户对安装没有特定的要求，但有特定的安装环境要求
2	用户提出了安装和转化的要求，转化/安装指南被经过测试提供给用户。但是转化的影响对该应用不重要
3	用户提出了安装和转化的要求，转化/安装指南被经过测试提供给用户。转化的影响对该应用来说是重要的
4	除了 2　的要求之外，需要提供经过测试的自动化的安装和转化工具。
5	除了 3　的要求之外，需要提供经过测试的自动化的安装和转化工具。

12. 易操作性

易操作性指的是应用对运行的影响程度，如有效启动、备份和恢复规程的影响。易操作性是应用提供的一种特性，它最小化了手工操作的要求。

0	用户没有指定除正常备份程序外的其它特定操作
1	提供高效的启动、备份和恢复进程，但需要人手操作
2	提供高效的启动、备份和恢复进程，不需要人手操作（当作两项计算）
3	应用程序对磁带的需求最小化
4	应用程序对硬拷贝处理的需求最小化
5	程序设计成无人操作模式。无人操作模式的意思是除了启动和关闭之外，不需要对系统进行操作。程序的其中一个功能就是错误自动恢复。

13. 多场地

多场地指应用系统经特殊设计、开发可以在多个组织、多个地点应用的程度。

0	用户需求不含多场地和组织的要求
1	考虑了多场地的要求，但是设计要求应用在不同的场地使用相同的软硬件环境

2	考虑了多场地的要求，但是设计要求应用在不同的场地使用类似的软硬件环境
3	考虑了多场地的要求，同时设计支持应用在不同的场地使用不同的软硬件环境
4	在 1　或者 2　的要求之上，提供了经过测试的多场地的文档和支持计划
5	在 3　的要求之上，提供了经过测试的多场地的文档和支持计划

2.2 功能点识别规则

2.2.1 EI/EO/EQ 识别规则

EI 中识别 FTR 规则

- 每一个 ILF 应该算做一个 FTR。
- 通过 EI 读取操作的每个 ILF 或 EIF 都应该被计算为一个 FTR。
- 即被 EI 维护又被读取的 ILF 仅计算一个 FTR。

EI 中识别 DET 规则

- 在 EI 的过程中，以用户角度识别的，通过应用系统边界输入系统内部的非重复的字段，那么该字段应算一个 DET。
- 如果在 EI 过程中，只要没有通过系统边界输入，就算它存在于系统内的一个 ILF 中，也不能算为一个 DET。

例如外贸订单系统中，订单的金额是被单价和数量自动计算的，那么金额是没有通过系统边界输入的，因此在 EI 操作中就不应该算做一个 DET。

- 在应用程序的 EI 操作时，系统提示的错误信息或完成操作的信息，应该被分别计算为一个 DET。

例如在网站注册用户信息时，由于输入错误系统会显示提示信息，那么这些提示信息应该被逐个计算为一个 DET。

当 EI 操作完成时系统提示并显示出来的信息，应该被计算为 DET。

- 在 EI 操作中如果遇到主外键的字段，应该算作一个 DET。

EO 和 EQ 计算 FTR 的规则

- 通用规则：每个在 EO/EQ 处理过程中读取的 ILF 和 EIF 算一个 FTR
- EO 额外的 FTR 计算规则

在 EO 处理过程中每个被维护的 ILF 算一个 FTR

在 EO 处理过程中既被读取又被维护的 ILF 算一个 FTR

EO 和 EQ 计算 DET 的通用规则

- 用户可识别的非重复的字段，进入应用边界并且指明处理什么，何时处理或处理方式，并且由 EO/EQ 返回或产生，那么这样的每个字段算一个 DET

例如在报表中的每个字段都是一个 DET

- 在应用边界内以用户角度识别的，非重复字段算一个 DET。

例如在报表上起到解释或备注作用的文字信息，不管它是一个字、一个词或一段话，都当作一个 DET

例如某种编号或日期，就算它被物理存储在不同字段中，但从用户角度来看是一个整体的信息，因此被算作一个 DET

例如在饼图中百分比和分类算作不同的 DET。

- 在 EO 或者 EQ 操作中，如果对系统进行输入或读取操作时，相同的字段只计算一个 DET。

例如在报表查询时，输入的字段在报表上也有显示，那么将算作同一个 DET

- 在应用程序的 EO 或 EQ 操作时，系统提示的错误信息或完成操作的信息，应该被计算为 DET。

例如用户查询一个列表时被拒绝，那么拒绝的提示信息就算为一个 DET。

- 在 EO 或 EQ 操作中如果遇到主外键的字段，应该算作一个 DET。
- 如果在 EO 或 EQ 过程中，只要没有通过系统边界输入，就算它存在于系统内的一个 ILF 中，也不能算为一个 DET。

在公司发工资的时候，员工对应的状态信息被更新，但这个状态信息的更新是没有通过系统边界输入的，因此也不能算做一个 DET。

- 页面的标题等类似的信息不计算 DET
- 系统字段生成的记号不能被算作一个 DET。

例如：页码、位置信息、时间、上一页、下一页等信息。

DET 识别规则

用户可识别，非重复的字段，通过基本流程处理 ILF/EIF 时获得。

eg: 如果员工号在一个 ILF 或 EIF 中出现两次，一次是作为员工记录的主键，一个是作为家属信息的外键，员工号记为一个 DET.

当两个应用程序维护或引用相同的 ILF、EIF，但是各自使用不同的 DET，那么仅计算它使用到的 DET。

eg: 应用程序 A 使用到的地址信息包括：street address, city, state, zip code. 应用程序 B 可能把地址当做一个整体而未细化到个体，因此应用程序 A 计数有 4 个 DET，而应用程序 B 计数有 1 个 DET。

每个被用户用来和其他 ILF 或 EIF 建立关系的数据也是一个 DET。

eg: 在 HR 系统中，员工信息是一个 ILF，员工职位名称也算作是员工信息的一部分，被算作一个 DET，因为它可以把员工和职位联系起来，这类 DET 成为外键。

RET 识别规则

RET 是指一个 EIF/ILF 中用户可以识别的 DET 的集合。

如果把 DET 简单理解为字段的话，那 RET 就可以简单地理解为数据库中的表。

如：订单内部逻辑文件由于存在订单头和明细关联引用，RET 应该算 2 个

2.2.2 内部逻辑文件与外部接口文件

ILF 内部逻辑文件

内部逻辑文件是指一组以用户角度识别的，在应用程序边界内且被维护的逻辑相关数据或控制信息。ILF 的主要目的是通过应用程序的一个或多个基本处理过程来维护数据。

EIF 外部接口文件

外部接口文件是指一组在应用程序边界内被查询，但它是在其他应用程序中被维护的，以用户角度来识别的，逻辑上相关的数据。因此一个应用程序中的 EIF 必然是其他应用程序中的 ILF。EIF 的主要目的是为边界内的应用程序提供一个或多个通过基础操作过程来引用的一组数据或信息。EIF 所遵循的规则：

从用户角度出发识别的一组逻辑数据。

这组数据是在应用程序外部，并被应用程序引用的。

计算功能点的这个应用程序并不维护该 EIF

这组数据是作为另一个应用程序中的 ILF 被维护的。

ILF 和 EIF 复杂性计算

ILF 和 EIF 的复杂性是取决于 RET（Record element type）和 DET（Data element type）的数量。DET 是一个以用户角度识别的，非重复的有业务逻辑意义的字段。

DET 计算的规则：

● 通过一个基本处理过程的执行，对 ILF 进行维护或从 ILF/EIF 中返回一个特定的、用户可识别的、非重复的字段，那么每个这样的字段算一个 DET。

例如：添加一个外贸订单时需要保存“订单号码、订单日期、地址、邮编”，那么对于 ILF 订单来说它的 DET 就是 4 个。

例如：保存订单时还会保存订单的明细，订单的明细往往作为一个子表进行保存，那么“订单号码”在主表和子表中都同时存在（主外键），但以用户角度来识别时，存盘操作是一个最小的单位，那么订单号码只能算做一个 DET。

● 当两个应用程序维护和/或引用相同的 ILF/EIF，但是每个应用程序分别维护/引用它们相应的 DET 时，这些 DET 在这两个应用程序的维护或引用中将单独计算。

例如一个应用程序的两个“Elementary Process”基本处理过程都需要使用到“地址”的信息，地址的信息又可以细分为“国家、城市、街道、邮编”。那么对于其中一个基本处理过程来说，他将整个地址信息作为一个整体进行处理，那就只算一个 DET，另外一个基本处理过程使用每个地址的详细信息，那么 DET 就是 4 个。

RET 计算的规则如下：

RET 是指一个 EIF/ILF 中用户可以识别的 DET 的集合。如果把 DET 简单理解为字段的话，那 RET 就可以简单理解为数据库中的表。RET 在 ILF/EIF 中分为两种类型：可选的（Optional）和必选的（Mandatory）。计算 RET 的规则为以下两点：

● 在一个 ILF/EIF 中每一个可选或必选的集合都被计算为一个 RET。或者

● 如果一个 ILF/EIF 没有子集合，则 ILF/EIF 被计算为一个 RET。

例如：在外贸订单系统中添加一个订单时会保存“订单信息、客户的 ID、部门的 ID”。

那么订单系统 ILF 中 RET 为：

1、 订单信息（必选的）

2、 客户信息（必选的）

3、 部门信息（可选的）因此 ILF 中 RET 的个数为 3 个。

外部输入 EI 的 DET 识别规则：

规则 1：在 EI 的过程中，以用户角度识别的、通过应用系统边界输入系统内部的非重复字段，算作一个 DET。

规则 2：在 EI 的过程中，只要没有通过系统边界输入，即使它存在于系统内的一个 ILF 中，也不能算为一个 DET。

例如：在外贸订单系统中，订单的金额是被单价和数量自动计算的，那么金额是没有通过系统边界输入的，因此在 EI 操作中不能算作一个 DET。

规则 3：EI 操作中系统提示的错误信息或完成的操作信息，应该分别算作一个 DET。

例如：在网站注册用户信息时，如果输入错误，系统会显示提示信息，那么这些提示信息应该被逐个计算为一个 DET；而当 EI 操作正确完成时，系统提示并显示出来的信息，也应该被计算为一个 DET。

外部输入 EI 的 FTR 识别规则：

规则 1：它所维护的每一个 ILF 算做一个 FTR。

规则 2：通过 EI 读取的每个 ILF/EIF 算作一个 FTR。

规则 3：既被 EI 维护又被读取的 ILF 仅计算一次。